AE646 Stage 3 (Final) Presentation

Optimising a Fourier Neural Operator for Parametric Darcy Flow

Recipe, architecture, data, resolution, latency, and comparison with a finite-volume solver

Team: PINNacles (Aryamann Srivastava, Varun Sathaye, Atishay Jain, Vedant S. Tiwari)

Recap and Stage 3 Questions

Learning the operator kappa -> u of -div(kappa grad u) = 1, on real PDEBench data

Where Stage 2 ended

- Real PDEBench pipeline: 900 / 100 / 200 split at 64x64, EDA, preprocessing and baseline ablations
- MLP (42 M): 0.082 mean rel. L2; FNO (4.7 M): 0.052
- Speed-up claim vs an FDM solver (1030 ms)

Stage 3 questions

- How much of the FNO error was optimisation, and how much capacity?
- Which architecture choices matter? How much data? Which resolution?
- What does inference cost, and how does the FNO compare with a proper finite-volume solver?
- What does the residual error mean physically?

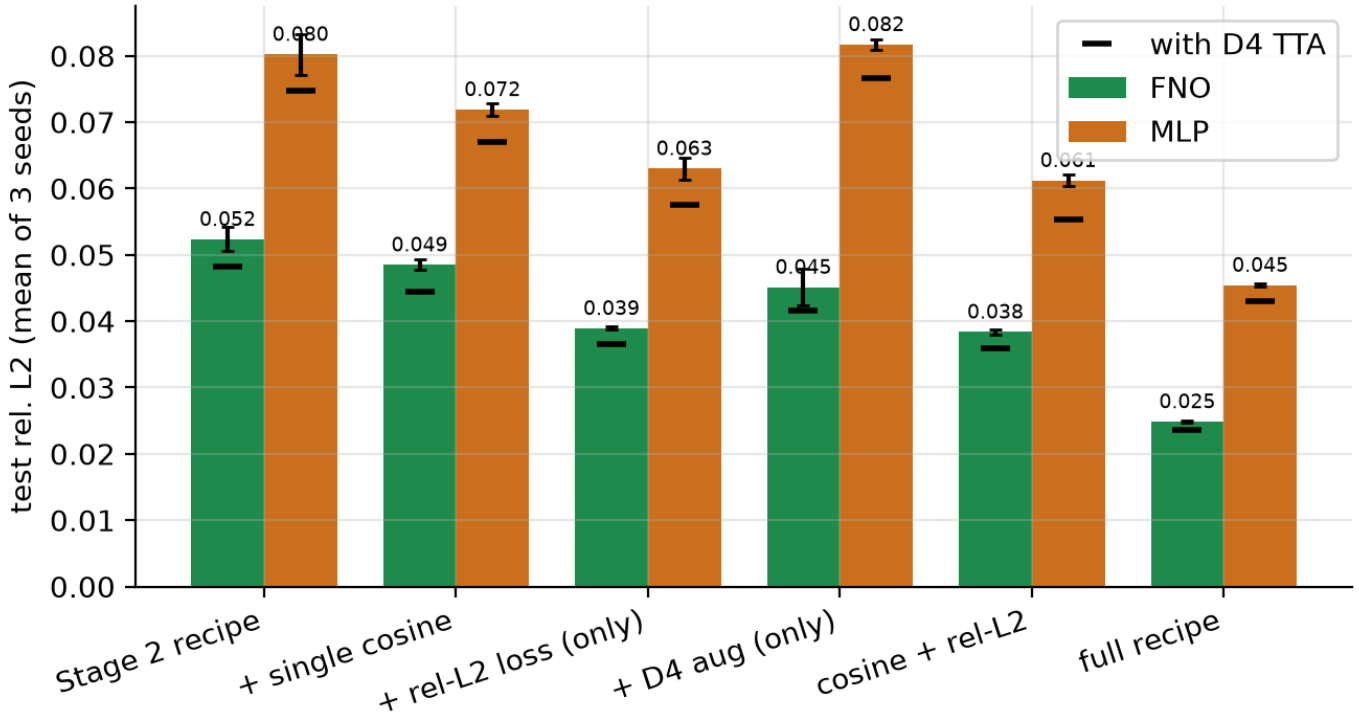
Experimental Protocol

Same split as Stage 2; every choice made on validation data only

- Data: 900 train / 100 val / 200 test real fields (seed 42); 8,800 further real samples used only for the data-scaling study
- Metric: per-sample relative L2 error in physical units, mean over test samples; test set evaluated once per run
- Reference step budget = Stage 2 (100 epochs = 5,700 steps, batch 16); seeds 0-4; sd over seeds reported
- Symmetry: data are D4-invariant; augmentation applied at 128x128 before sub-sampling (valid), test-time averaging over {identity, transpose}
- Baseline given the same effort: MLP tuned on the same footing (capacity sweep, validation selection)
- 142 training runs on one GPU workstation; all configs and metrics stored per run

Finding 1: The Training Recipe Limited Stage 2

Same models, same 5,700 steps, 3 seeds - only the recipe changes

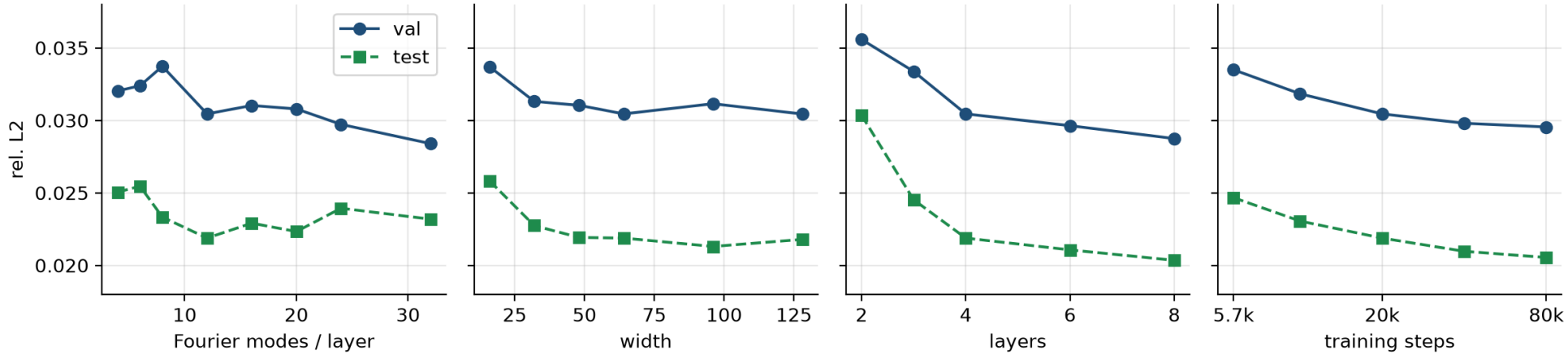


Result

- Stage 2 recipe: MSE loss; cosine LR restarted every 200 steps
- Relative-L2 loss: -26%. Single cosine: -7%. D4 augmentation on top: -35%
- FNO 0.0524 -> 0.0249 (2.1x lower); MLP 0.0802 -> 0.0454
- Gains interact: D4 alone does not help under the old recipe

Finding 2: Architecture Choices Matter Little

One-factor sweeps around (12 modes, width 64, 4 layers); selection on validation error



What we learned

- Modes 4 to 32 (0.5 M to 34 M parameters): only 0.022-0.025 test error; width >= 32 saturates
- Depth and training length help slowly: 2 -> 8 layers 0.030 -> 0.020; 5.7k -> 80k steps 0.0247 -> 0.0205
- Learning rate, weight decay, batch size: < 0.001 effect. Validation picks deeper, not wider: (12, 64, 8), 9.5 M

Final Models: FNO vs Tuned MLP

60,000 steps, full recipe, 5 seeds, 200 held-out test fields

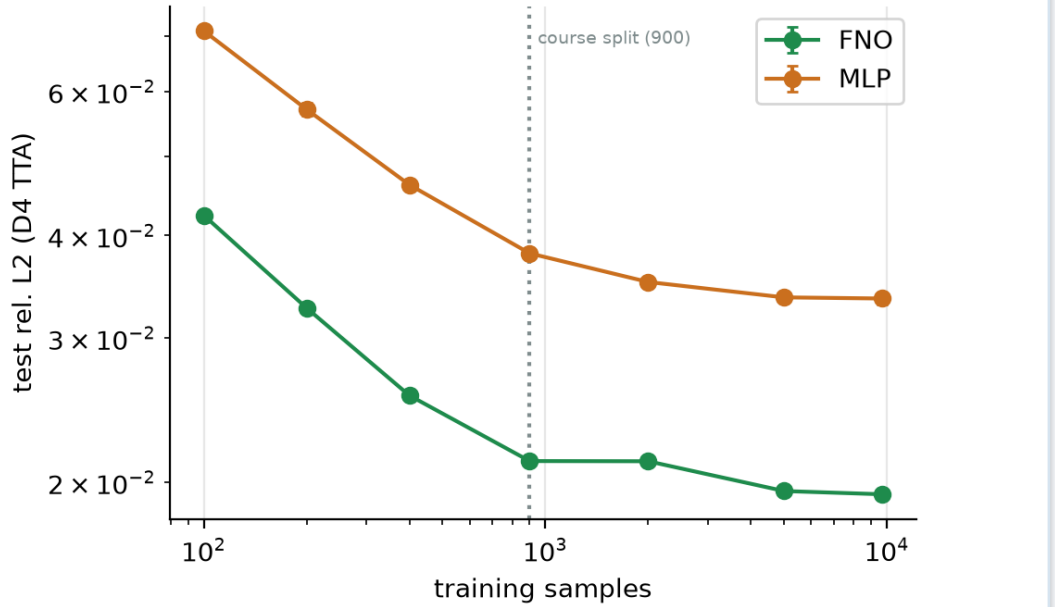
Model	Params	Mean rel. L2	Median	With TTA
Stage 2 MLP	42.0 M	0.0820	0.0695	-
Stage 2 FNO	4.7 M	0.0521	0.0398	-
Final MLP (3x4096)	100.7 M	0.0415 +- 0.0009	0.0272	0.0386
Final FNO (w64, L8)	**9.5 M**	**0.0202 +- 0.0004**	**0.0070**	**0.0200**

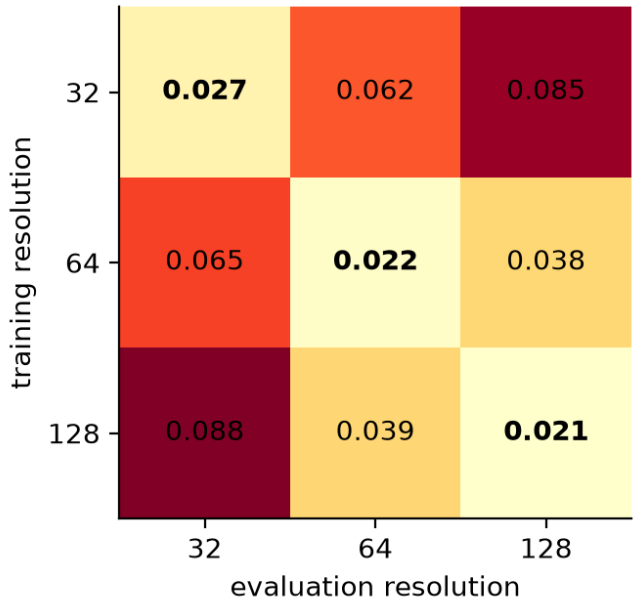
Take-aways

- FNO error 2.05x lower than an equally tuned MLP (Stage 2 gap: 1.6x) with 10.6x fewer parameters; 2.6x lower than the Stage 2 FNO
- Median FNO error is 0.7%: the mean is dominated by a small tail (next slides)
- Training the final FNO: about 32 minutes on one shared GPU

Finding 3: Data Scaling and Resolution

FNO vs MLP with 100 to 9,700 samples; train-vs-evaluation resolution matrix





- FNO with 400 samples beats the MLP with 9,700; FNO plateaus near 0.0195 for >= 5,000 samples
- Zero-shot transfer 64 -> 128: 0.038 (native 128: 0.021); 64 -> 32: 0.065: useful, but 1.7-3x worse than native resolution

Finding 4: Comparison with a Finite-Volume Solver

Correction to Stage 2: the old FDM reference was inefficient - the 1443x / 7740x speed-ups are withdrawn

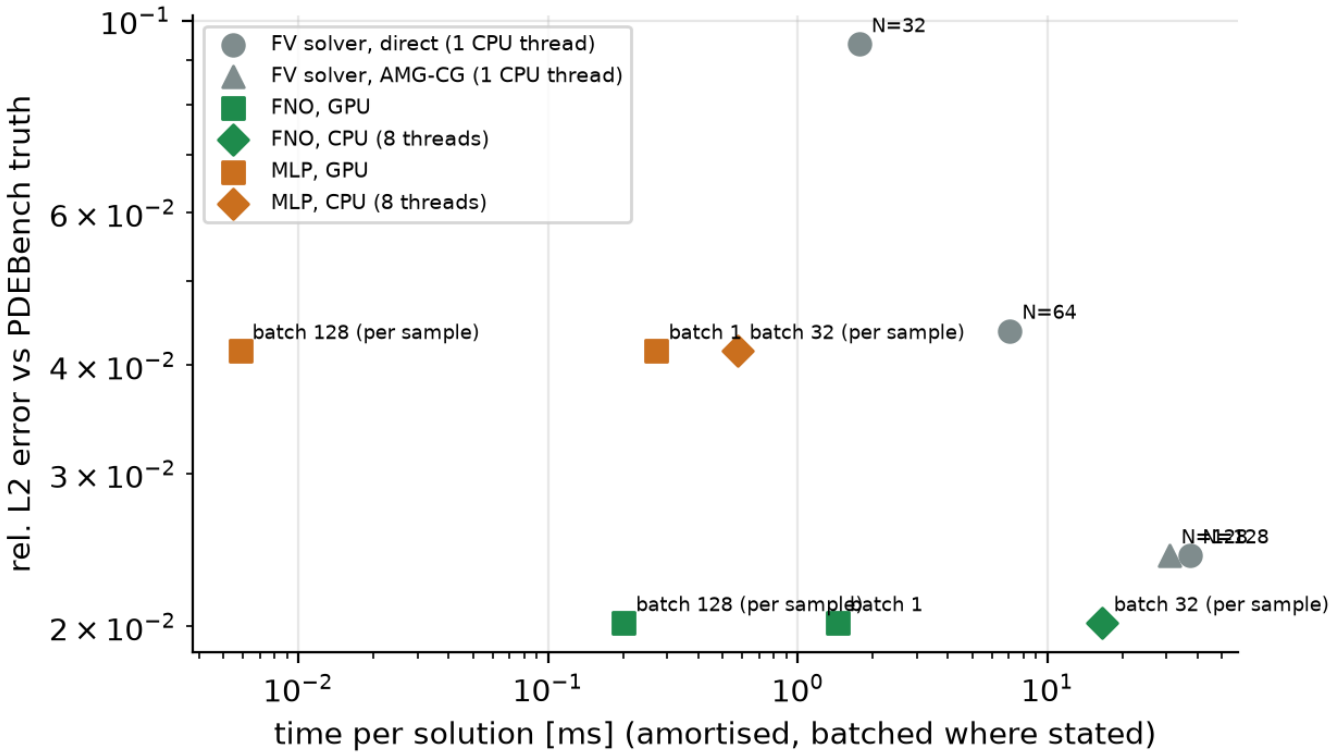
Grid	Calibrated FV	Error vs truth	Direct LU	AMG-CG
32 x 32	harmonic, b = 0.7	0.094	1.8 ms	5.5 ms
64 x 64	harmonic, b = 0.8	0.044	7.1 ms	11.6 ms
128 x 128	arithmetic, b = 1.05	0.024	37.4 ms	30.7 ms
FNO, 64 x 64 (GPU)	**9.5 M parameters**	**0.020**	**1.4 ms (b=1)**	**0.20 ms (batched)**

How to read it

- Vectorised solver; boundary distance b calibrated on validation data (PDEBench's truth behaves as if the Dirichlet boundary were ~1 cell outside the array)
- At equal resolution the FNO agrees with the data 2.2x better; partly because it learns the generator's own discretisation
- Single CPU thread: FNO 37 ms at batch 1 - NOT faster than the 64^2 solver (7 ms); the gain is GPU batching

Inference Cost: Error vs Time per Solution

CUDA-event timing on an idle GPU; CPU numbers from the workstation CPU (indicative)

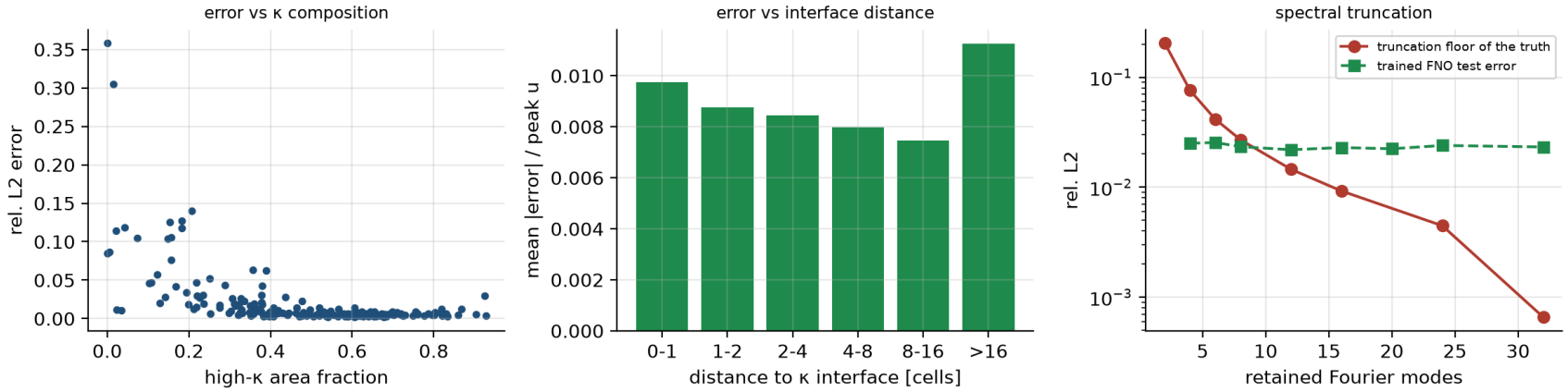


Latency at 64 x 64

- FNO (9.5 M): 1.44 ms at batch 1, 0.20 ms per sample at batch 128
- 5x (batch 1) to 35x (batched) faster than the 64^2 solver, and 26-187x faster than the 128^2 solver
- MLP: 0.006 ms/sample but 2x worse accuracy
- TF32 gives no gain; bfloat16 unsupported for complex FFT
- Break-even vs the 128^2 solver: ~5e4 solves

Physical Interpretation

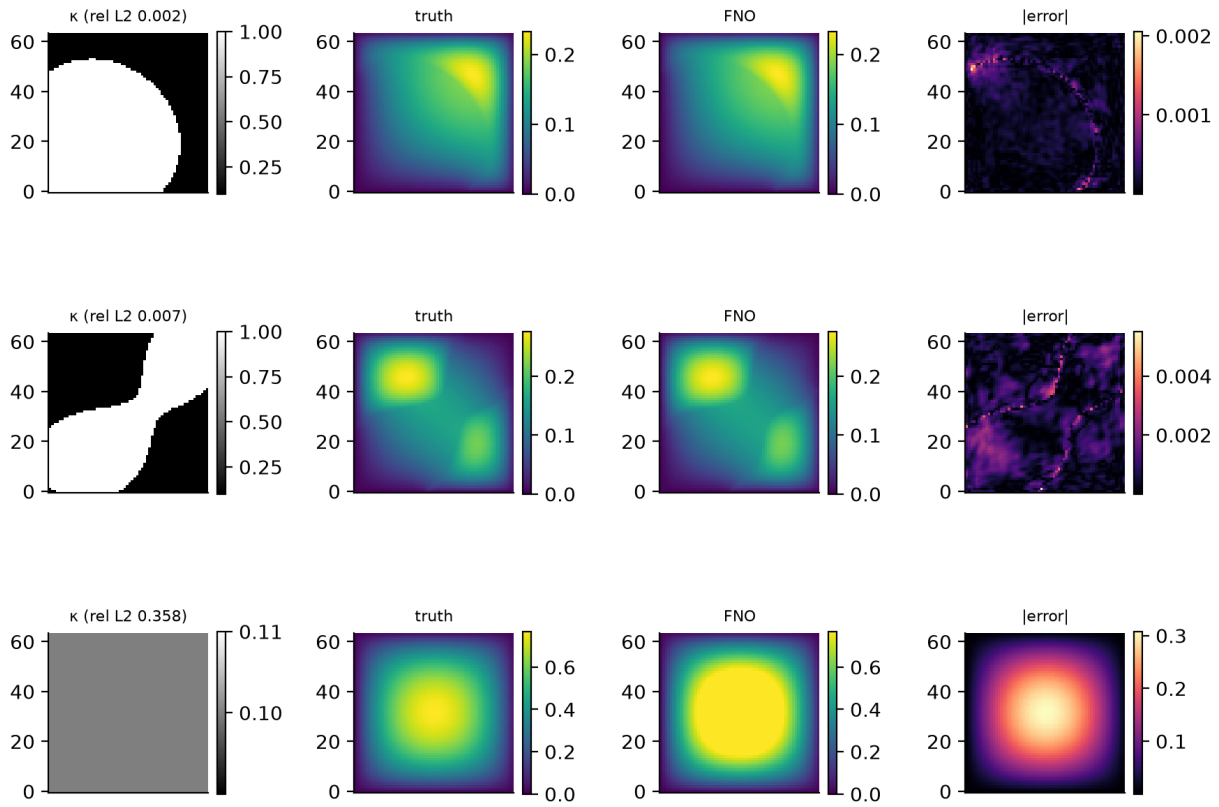
Final FNO on the 200 test fields



- Physics respected: PDE residual within 6% of the truth's own; mean pressure to 1.7%; boundary ring ratio 0.1157 vs 0.1159; 1.5e-5 of pixels negative
- Error is NOT concentrated at interfaces (0.0098 next to them vs 0.0075 at 8-16 cells); largest in large low-kappa regions with high pressure (r = 0.61 with peak u)
- Spectral: truth needs ~12 modes (1.5% truncation error), but a 4-mode FNO still reaches 2.5%: the bypass, nonlinearity and depth regenerate high frequencies

Best, Median and Worst Test Sample

kappa | truth | FNO | absolute error



Reading the figure

- Best 0.002, median 0.007
- Worst 0.358: uniform kappa = 0.1; correct shape, saturated amplitude
- The two uniform fields have errors 0.358 and 0.085; without them the mean falls from 2.0% to 1.8%
- Since u ~ 1/kappa for uniform kappa, a scale-normalised formulation could remove this failure (not tested)

Limitations and Honest Notes

What the results do and do not show

- One PDE, one dataset, binary kappa, f = 1; the plateau and scaling laws may not transfer
- Top architectures differ by less than seed noise; single seed per sweep point (final models: 5 seeds)
- Stage 2 absolute errors reflected a sub-optimal recipe (same for both models, so the comparison was fair); Stage 2 speed-up figures withdrawn
- Solver was calibrated to the dataset: the comparison measures agreement with PDEBench, not fidelity to the continuous PDE; single-thread, uncompiled solver
- Timings on a shared machine: CPU numbers are indicative; GPU timing on an idle GPU

Summary

Recipe	Relative-L2 loss + single cosine + D4 augmentation halve the FNO error at equal cost (0.052 -> 0.025)
Final	FNO 9.5 M: 0.0202 +- 0.0004 vs tuned MLP 100.7 M: 0.0415 +- 0.0009 (5 seeds): 2.05x, 10.6x fewer parameters
Limits	Insensitive to modes/width; plateaus near 2% for >= 5,000 samples; zero-shot resolution transfer is moderate
Solver	2.2x more accurate than a calibrated FV solver at 64x64 and 5-35x faster on GPU; not faster on one CPU core
Physics	Respects BC, positivity, discrete PDE; residual error sits in uniform / low-contrast, high-amplitude fields

Thank you - Questions welcome